A Bayesian Write-Audit-Publish (WAP) Architecture for High-Volume ETL Pipelines: Decoupled Statistical Validation with Human-in-the-Loop Governance

Tom K. Chang
tomc30098@gmail.com

Abstract

Data validation within CI/CD pipelines for population-scale administrative datasets presents two compounding challenges: the computational cost of scanning millions of records per pipeline run, and the scarcity of historical baselines when only a handful of annual data epochs exist. Frequentist hypothesis tests reject the null for negligible effect sizes at population scale ($N > 10^6$), producing excessive false alarms. Hard-coded thresholds require manual tuning and do not adapt as data evolve.

We describe a two-stage validation architecture for high-volume, low-frequency (HVLF) administrative data systems. In Stage 1, a single $O(N)$ scan distills millions of records into sufficient statistics persisted as a kilobyte-scale pipeline artifact. In Stage 2, a Normal-Inverse-Gamma conjugate model scores each metric's year-over-year log-ratio against a posterior predictive Student-t distribution, operating in $O(1)$ per metric. Hierarchical pooling across 50+ metrics calibrates a shared variance prior using the median of per-metric sample variances.

An adaptive prior-weighting scheme tiers the prior precision κ_0 by historical depth, increasing detection sensitivity for newly introduced or recently reset metrics while preserving data-dominant scoring for established ones. A deterministic materiality filter suppresses false alarms on small-count metrics using interpretable step-function thresholds tied to historical depth.

A human-in-the-loop (HITL) circuit breaker integrates with CI/CD infrastructure to block anomalous data from production, supporting four typed override mechanisms: anomaly exclusion, regime-change truncation, acknowledged outliers, and transition-state momentum priors. The override taxonomy supports structured lifecycle patterns—notably a `TRANSITION_STATE` → `REGIME_CHANGE` sequence for multi-year policy transitions. A cross-environment artifact-injection protocol relays pipeline context metadata from the cloud agent to a local Docker-hosted review application, enabling the auditor to write overrides and re-trigger the failed pipeline stage from a single interface without manual identifier transcription.

The architecture is evaluated on a deployment processing millions of records across 80+ monitored metrics, providing a scalable blueprint for regulated industries where audit trails and human oversight are non-negotiable.

1 Introduction

Administrative datasets in education, government, healthcare, and insurance share a common profile: they are high-volume (millions of records per epoch) yet low-frequency (produced annually or quarterly). We refer to this as the HVLF regime. Integrating statistical validation into CI/CD pipelines for HVLF data exposes three failure modes:

Memory symmetry. Distribution-comparison tests (Kolmogorov–Smirnov, chi-square goodness-of-fit) require both current and reference datasets in memory simultaneously, risking out-of-memory failures in resource-constrained CI containers.

The large-sample fallacy. At $N > 10^6$, standard hypothesis tests reject the null for negligible effect sizes. In production deployment, proportion z-tests and chi-square tests flagged the majority of metrics as statistically significant despite substantively trivial year-over-year changes.

Temporal sparsity. With only 3–5 annual epochs, frequentist control charts cannot reliably estimate process variance. Standard Statistical Process Control requires 20–25 subgroups for parameter estimation [Montgomery, 2009].

The governance gap. Beyond statistical methodology, a practical gap separates the two communities who must collaborate: data engineers who build pipelines lack statistical training, while data scientists who understand Bayesian inference rarely engage with CI/CD infrastructure. When a pipeline blocks on a statistical anomaly, the remediation loop—reviewing the flag, classifying its cause, writing an override, and re-triggering only the affected stage—typically spans multiple tools, manual copy-paste of run identifiers, and undocumented tribal knowledge.

Contributions. Our contribution is not a novel statistical method—the Normal-Inverse-Gamma conjugate family is textbook material [Gelman et al., 2013]. Nor is the CI/CD pipeline architecture novel. The contribution is the integration of known statistical and engineering components into a cohesive governance architecture that addresses this gap. The architecture:

1. Decouples $O(N)$ computation (Stage 1) from $O(1)$-per-metric governance (Stage 2) via compressed sufficient-statistics artifacts;
2. Uses log-ratio transforms for symmetric, additive year-over-year comparisons;
3. Handles cold-start scenarios via hierarchical priors with robust median pooling;
4. Implements adaptive prior weighting (κ_0 tiering) that increases detection sensitivity for low-history metrics while preserving data dominance for established ones;
5. Implements a circuit breaker with four typed override mechanisms, a persistent audit trail, and structured lifecycle patterns for multi-year transitions;
6. Bridges the cloud-agent and local-auditor environments through an artifact-injection protocol that relays pipeline context metadata, enabling single-interface override entry and selective stage re-triggering;
7. Supports materiality-aware scoring via deterministic step-function thresholds that are immediately interpretable by non-technical stakeholders;
8. Operates in configurable enforcement modes (preliminary soft-pass vs. final hard-block) aligned with branch-protection policies.

2 System Architecture

Figure 1 presents the end-to-end workflow. The architecture instantiates the Write-Audit-Publish (WAP) pattern [Databricks, 2023] with two key extensions: (i) the Write stage publishes *compressed sufficient statistics* rather than full datasets, yielding a "Write-Compress-Audit-Publish" variant; and (ii) the Audit stage employs adaptive Bayesian scoring with a closed HITL remediation loop rather than static rule-based checks.

2.1 Stage 1: Sufficient Statistics Extraction (Write Phase)

Let D_t denote the raw dataset for epoch t, with $|D_t| = N_t$ records. For each metric $m \in \{1, \ldots, M\}$, Stage 1 computes

$$S_t^{(m)} = f_m(D_t), \tag{1}$$

where f_m is a metric-specific aggregation function producing counts, proportions, or distributional moments. The summary vector $\mathbf{S}_t$ is persisted as a JSON artifact (typically $< 10\,\text{KB}$). Stage 2 operates exclusively on these aggregates.

In production, Stage 1 is implemented as a pytest suite processing millions of records. A critical architectural constraint is preventing Out-Of-Memory (OOM) failures on resource-limited CI/CD build agents. While the time complexity of Stage 1 is strictly $O(N)$, we bound the memory complexity to $O(1)$ with respect to N by processing data partitions sequentially and enforcing aggressive garbage collection after each intermediate transformation. This ensures the pipeline scales seamlessly from 10^6 to 10^8 records without requiring vertical scaling of

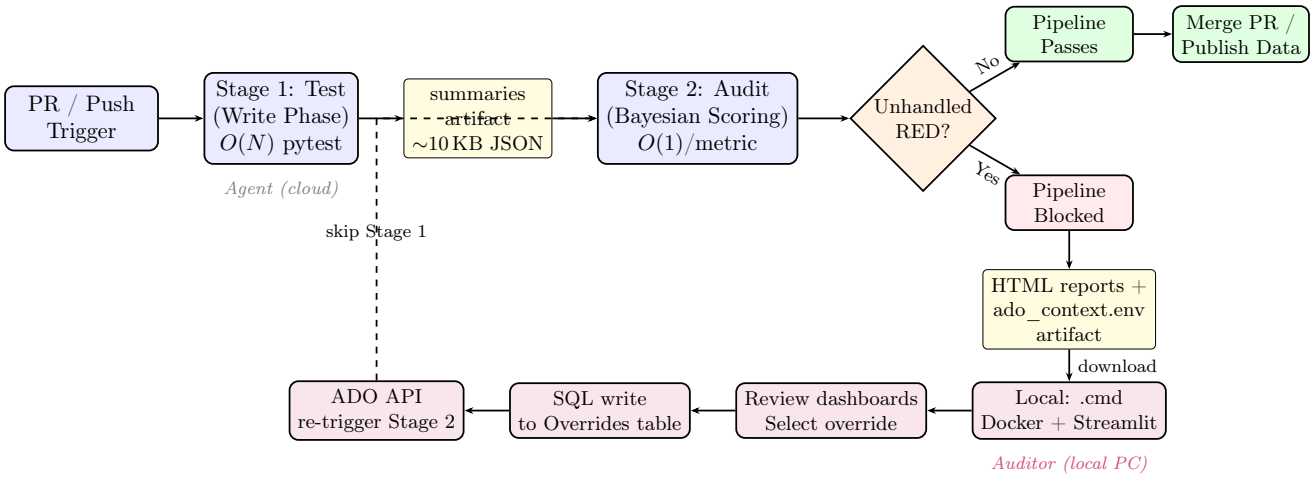

Figure 1: End-to-end architecture. Stage 1 runs on the CI agent and publishes compressed sufficient statistics. Stage 2 performs Bayesian scoring and, on unhandled anomalies, blocks the pipeline and publishes an artifact bundle containing HTML dashboards and pipeline-context metadata. The auditor downloads the bundle, reviews reports in a local Docker-hosted Streamlit application, writes overrides directly to the database, and re-triggers Stage 2 via the Azure DevOps API—all from a single interface, with no manual identifier transcription. Dashed arrow indicates selective stage re-triggering.

the CI infrastructure. At completion, Stage 1 publishes a compressed tarball (`summaries.tar.gz`) containing the $O(1)$-sized per-metric JSON files.

This design serves a dual purpose within the same fiscal year: Stage 1 validates same-year code correctness across multiple preliminary data versions (schema tests, logic tests, referential integrity), while Stage 2—activated for the final annual release—validates year-over-year statistical consistency. Branch-protection policies enforce this distinction: developers may self-merge for preliminary runs, but the final merge to `main` requires Stage 2 passage.

2.2 Stage 2: Bayesian Governance (Audit Phase)

Stage 2 downloads the sufficient-statistics artifact, loads historical baselines from a persistent ground-truth table, and applies the Bayesian scoring model described in Section 3. The scoring module is framework-agnostic: it requires only a SQL backend (for ground truth and overrides), a Python runtime, and a CI/CD platform with conditional artifact publishing.

Circuit breaker. When Stage 2 identifies unhandled **RED** metrics, it terminates with a non-zero exit code, halting the pipeline. The HTML dashboard is published as a pipeline artifact regardless of exit status (using unconditional artifact publishing), ensuring reviewers can always inspect results even when the pipeline fails.

Enforcement modes. A runtime environment variable (`RUN_PHASE`) controls enforcement strictness. In `PRELIM` mode, **RED** flags generate dashboards but do not block the pipeline, supporting rapid iteration during development. In `FINAL` mode, any unhandled **RED** triggers a hard block. This allows teams to configure enforcement by lifecycle stage without modifying code.

2.3 Artifact-Injection Protocol

A key engineering contribution is the *artifact-injection protocol* that bridges the stateless cloud CI agent and the auditor's local workstation. The challenge is that the CI agent has access to pipeline context metadata (pull-request ID, pipeline ID, organization URL, run phase) that the auditor needs to re-trigger the pipeline, but this metadata is lost when the pipeline terminates.

Our solution exploits the pipeline artifact as a relay mechanism:

1. **Context capture.** Before Stage 2 exits, it writes CI environment variables (e.g., `System.PullRequest.PullRequestId`, `Build.BuildId`) into a plain-text file (`ado_context.env`) alongside the HTML dashboards.
2. **Artifact bundle.** The YAML pipeline publishes the entire bundle—dashboards, scoring results, and context file—as a downloadable artifact, regardless of exit status.
3. **Local hydration.** The auditor downloads and extracts the artifact. A `.cmd` launcher starts a Docker container hosting a Streamlit application. On startup, the application reads `ado_context.env`, hydrating itself with the PR identifier, pipeline ID, and other parameters.
4. **Single-interface remediation.** The Streamlit UI presents the Plotly dashboards, a selectbox for each flagged metric's override status, and a free-text notes field. On submission, it executes two actions: (a) writes the override to the `CIGroundTruth_Overrides` table via SQL authentication with write permissions; and (b) fires an Azure DevOps REST API call to retrigger the pipeline for the same PR, skipping Stage 1 (whose artifacts are already cached).

This protocol has three properties:

- **Zero manual transcription.** The auditor never copies PR numbers, pipeline IDs, or branch names. The artifact carries everything.
- **Security separation.** Stage 1 and Stage 2 run on the CI agent with read-only database credentials. Write credentials exist only in the local Docker container, never exposed to the CI environment.
- **Selective re-execution.** The API call skips Stage 1 (which is expensive, processing > 2.6M records) and re-runs only Stage 2 (which is $O(M)$ where $M \ll N$), saving compute and wall-clock time.

The reference implementation of the pipeline YAML, auditor Streamlit application, and Dockerfile is provided in Section A.

2.4 Override Reset Protocol

Within a fiscal year, multiple preliminary data versions may trigger Stage 2 re-evaluation. To prevent stale overrides from masking new anomalies in subsequent preliminary versions, the `.cmd` launcher includes a reset step: before opening the Streamlit UI, it truncates all overrides for the current fiscal year in the overrides table. This ensures each preliminary cycle begins from a clean override state, while final-year overrides (which have passed full review) are preserved by fiscal-year partitioning.

3 Statistical Methodology

3.1 Log-Ratio Transform

Rather than the standard proportional change $\delta_t = (S_t - S_{t-1})/S_{t-1}$, we use the log-ratio:

$$\lambda_t^{(m)} = \log\left(\frac{S_t^{(m)}}{S_{t-1}^{(m)}}\right). \tag{2}$$

The log-ratio offers three advantages over proportional change. First, *symmetry*: a doubling ($+100\%$) and a halving (-50%) map to $+0.693$ and -0.693 respectively, placing equal-magnitude changes at equal distances from zero. Second, *additivity*: consecutive log-ratios sum to the total log-ratio over the period, enabling coherent multi-year comparisons. Third, the Gaussian assumption underlying the Normal-Inverse-Gamma model is more tenable on the log scale, where multiplicative effects become additive.

Metrics with zero or negative values in either epoch are excluded from scoring, as the log-ratio is undefined.

3.2 Bayesian Model

Model specification. For each metric m, we model log-ratios as:

$$\lambda_t^{(m)} \mid \mu_m, \sigma_m^2 \sim \mathcal{N}(\mu_m, \sigma_m^2) \tag{3}$$

$$\mu_m \mid \sigma_m^2 \sim \mathcal{N}\left(\mu_0, \frac{\sigma_m^2}{\kappa_0}\right) \tag{4}$$

$$\sigma_m^2 \sim \text{Inv-Gamma}(\alpha_0, \beta_0) \tag{5}$$

Interpretation of $\mu_0 = 0$. It is important to distinguish the prior mean $\mu_0 = 0$ from a frequentist null hypothesis $H_0 : \mu = 0$. In hypothesis testing, $\mu = 0$ is a fixed parameter assumption tested against data; rejection is binary. In our Bayesian framework, $\mu_0 = 0$ is an initial belief that is *progressively overwhelmed* by data. The posterior mean

(Equation (9)) is a weighted average of prior and data:

$$\mu_n = \underbrace{\frac{\kappa_0}{\kappa_0 + T}}_{\text{prior weight}} \cdot \mu_0 + \underbrace{\frac{T}{\kappa_0 + T}}_{\text{data weight}} \cdot \bar{\lambda}. \tag{6}$$

At $T = 3$ with $\kappa_0 = 1$, the prior contributes only 25% of the posterior mean; by $T = 10$, less than 10%. A metric exhibiting consistent growth (e.g., log-ratios of $+0.10, +0.12, +0.09$) will see its posterior mean pulled toward $\bar{\lambda} \approx +0.10$, and only deviations from *that learned trend* will be flagged—not the trend itself. The system thus detects *departures from each metric's own historical pattern*, not departures from zero.

Hierarchical prior calibration. With M metrics but only T log-ratios each, per-metric variance estimation is unreliable. We pool across metrics:

1. For each metric m, compute $s^{2(m)}$ from its T historical log-ratios.
2. $s^2_{\text{pool}} = \text{median}(s^{2(1)}, \ldots, s^{2(M)})$.
3. Set $\mu_0 = 0$, $\alpha_0 = 5$, $\beta_0 = \alpha_0 \cdot s^2_{\text{pool}}$.

We use the median rather than the mean for robustness. In production, one metric (a subgroup undergoing definitional changes) exhibited year-over-year changes corresponding to log-ratios of $+3.28$ and $+0.88$, which would inflate a mean-based pooled variance by two orders of magnitude while leaving the median unaffected.

Adaptive prior weighting (κ_0 tiering). The prior precision κ_0 controls how quickly data overwhelms the prior. A fixed $\kappa_0 = 1$ assigns the prior the weight of one pseudo-observation, appropriate when $T \geq 2$. However, for metrics with limited history—newly introduced metrics or those recently reset via `REGIME_CHANGE`—$\kappa_0 = 1$ produces an overly diffuse posterior predictive, making anomaly detection ineffective.

We tier κ_0 by historical depth T:

$$\kappa_0(T) = \begin{cases} 3.0 & T = 0 \quad \text{(defensive; currently unreachable)} \\ 2.0 & T \leq 3 \quad \text{(early history or post-regime)} \\ 1.0 & T > 3 \quad \text{(data-dominant)} \end{cases} \tag{7}$$

The $T = 0$ case is defensive: the current pipeline's inner-join structure guarantees $T \geq 1$, but the guard accommodates future extensions that may introduce metrics without historical anchors. At $T \leq 3$, $\kappa_0 = 2$ assigns the prior the weight of two pseudo-observations, tightening the posterior predictive scale and increasing sensitivity to anomalies during the critical early-history period. For $T > 3$, the prior contributes less than 20% of the posterior mean, making the specific κ_0 value inconsequential—all three settings converge (Figure 2).

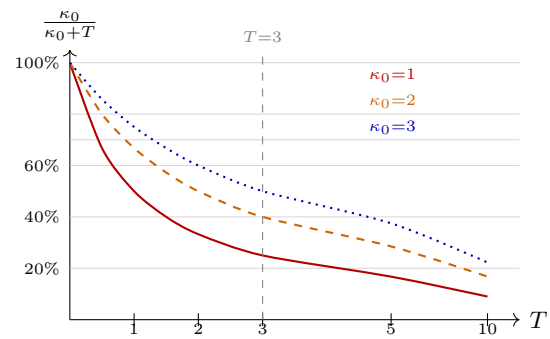

Figure 2: Prior weight $\kappa_0/(\kappa_0 + T)$ as a function of historical depth. All three κ_0 settings converge by $T \geq 3$, confirming that the tiering affects only low-history metrics.

This tiering interacts naturally with the `REGIME_CHANGE` override (Section 4.1): when a regime change truncates history to $T = 1$, the elevated $\kappa_0 = 2$ automatically takes effect, providing tighter monitoring during the post-reset period without requiring separate configuration.

$\alpha_0 = 5$ is a weakly informative choice; sensitivity to this hyperparameter is discussed in Section 9.

Posterior update. The conjugate structure yields closed-form posteriors:

$$\kappa_n = \kappa_0 + T \tag{8}$$

$$\mu_n = \frac{\kappa_0 \mu_0 + T\bar{\lambda}}{\kappa_n} \tag{9}$$

$$\alpha_n = \alpha_0 + T/2 \tag{10}$$

$$\beta_n = \beta_0 + \tfrac{1}{2}\text{SSE} + \frac{\kappa_0 T}{2\kappa_n}(\bar{\lambda} - \mu_0)^2 \tag{11}$$

Posterior predictive and scoring. The posterior predictive distribution is:

$$\lambda_{\text{new}} \sim t_{2\alpha_n}\left(\mu_n,\ \frac{\beta_n(\kappa_n + 1)}{\alpha_n \kappa_n}\right). \tag{12}$$

We compute the two-tailed extreme probability p_{extreme} and assign: **RED** if $p < 0.01$, **YELLOW** if $p < 0.05$, **GREEN** otherwise.

The Student-t distribution provides thicker tails when T is small, yielding conservative behavior during early deployment that automatically sharpens as historical depth increases—without any threshold adjustment.

3.3 Materiality-Aware Scoring

Proportional and log-ratio changes are sensitive to small denominators. A change from 14 to 16 organizational units produces a log-ratio of +0.134 and a p-value of 0.001, yet represents only 2 additional units in a region with inherently small counts.

We introduce a deterministic step-function materiality threshold:

$$\tau(T) = \begin{cases} 30 & T < 2 \\ 10 & 2 \leq T < 4 \\ 5 & T \geq 4 \end{cases} \tag{13}$$

If $\min(S_t, S_{t-1}) < \tau(T)$, the metric's flag is appended with `_LOW_CONF`. The `LOW_CONF` suffix is informational: it appears on the dashboard for reviewer awareness but does *not* block the pipeline, even when the underlying Bayesian score is **RED**.

We chose a deterministic step function over an exponential decay (e.g., $\tau_{\text{base}} \cdot e^{-\gamma(T-2)}$) for interpretability: any stakeholder can verify the threshold by reading three numbers. The comparison uses $\min(S_t, S_{t-1})$ rather than S_t alone, ensuring that metrics with small base counts in either epoch are correctly identified—a metric jumping from 8 to 12 should be flagged as low-confidence despite its current value exceeding the threshold.

4 Human-in-the-Loop Override Taxonomy

The statistical scoring is embedded within a governance loop following the WAP pattern. A persistent database table (`CIGroundTruth_Overrides`) supports four override types, each with distinct statistical treatment and a recorded audit trail (timestamp, author, free-text notes).

4.1 Override Types

(1) `IGNORE_ANOMALY`**.** Confirmed data error. The metric's row is filtered at the SQL layer *before data enters the scoring module*. The erroneous delta never participates in prior calibration or scoring. This is the strongest exclusion.

(2) `REGIME_CHANGE`**.** Policy or definitional change (e.g., reclassification of school types). All historical data for the affected metric prior to the regime year is truncated, resetting the baseline. Future scoring operates only on post-regime epochs. Because truncation reduces T, the κ_0 tiering (Equation (7)) automatically elevates the prior precision, providing tighter monitoring during the post-reset period.

(3) `ACKNOWLEDGED_OUTLIER`**.** The deviation is real, understood, and accepted. The data point is *retained* in all future likelihood calculations, naturally widening the posterior variance for that metric. This is appropriate for one-time events (enrollment spikes, school openings/closings) where the cause is documented but the data itself is correct. By keeping the outlier in history, the model learns that this metric can exhibit occasional large swings, reducing sensitivity to similar future events.

(4) `TRANSITION_STATE`**.** The metric is undergoing a multi-year structural shift (e.g., gradual enrollment growth following a new program or policy rollout). The prior mean μ_0 is replaced with the most recent historical log-ratio, implementing a momentum-based prior:

$$\mu_0 \leftarrow \lambda_T^{(m)}, \quad \kappa_0 \leftarrow \max(\kappa_0, 2.0). \tag{14}$$

The elevated κ_0 ensures the momentum estimate receives at least 33% of the posterior weight at $T = 4$ (vs. 20% under the default $\kappa_0 = 1$), preventing historical stable periods from fully diluting the momentum signal. This is an empirical Bayes heuristic; we acknowledge its ad-hoc nature and recommend it only when domain knowledge supports a trend assumption.

4.2 Override Lifecycle Patterns

The four override types are not independent choices; they form structured lifecycle sequences.

The most common pattern addresses multi-year policy transitions:

1. **Year 1 (shock):** A policy change causes a large metric shift. The system flags **RED**. The auditor applies `TRANSITION_STATE`, signaling that continued movement is expected.
2. **Year 2 (continued drift):** The metric continues to shift, possibly at a reduced rate. The momentum prior ($\mu_0 = \lambda_T$) absorbs the continued movement. If the metric still flags **RED**, the auditor reapplies `TRANSITION_STATE`.
3. **Year 3 (stabilization):** The metric reaches its new equilibrium. The auditor applies `REGIME_CHANGE`, truncating all pre-transition history. The κ_0 tiering automatically elevates to 2.0 (since T resets to 1), providing heightened sensitivity at the new baseline.
4. **Year 4+ (normal operation):** As post-regime history accumulates, κ_0 decays to 1.0 and the model operates in data-dominant mode with the new baseline.

When uncertain whether a metric has stabilized, auditors should default to `TRANSITION_STATE` rather than `REGIME_CHANGE`. If the metric has in fact stabilized, the momentum prior ($\mu_0 = \lambda_T$ where $\lambda_T \approx 0$) will be close to the default prior ($\mu_0 = 0$), producing negligible distortion. The reverse error—premature `REGIME_CHANGE` followed by continued drift—discards potentially valuable history and may require another `TRANSITION_STATE` the following year.

All override decisions are permanently recorded in `CIGroundTruth_Overrides` with fiscal year, metric, category, status, notes, and timestamp, providing a complete audit trail even after `REGIME_CHANGE` truncates the active history.

4.3 Selective Assimilation

The override taxonomy induces a selective assimilation protocol: `IGNORE_ANOMALY` data never enters the system; `ACKNOWLEDGED_OUTLIER` data remains in history and naturally widens variance; `REGIME_CHANGE` truncates history; `TRANSITION_STATE` modifies the prior mean. Only unoverridden **GREEN** and **YELLOW** metrics are fully assimilated into future priors without modification. This ensures that the prior evolves to reflect the system's normal operating regime, not its exceptions.

Table 1 summarizes the parameter effects of each override type.

Table 1: Override parameter effects on the Bayesian scoring model.

Override	Prior μ_0	Prior κ_0	History	Gate
`ACK_OUTLIER`	0	default	Kept (widens var.)	Pass
`TRANSITION`	λ_T (momentum)	$\max(\kappa_0, 2)$	Kept (all years)	Pass
`REGIME_CHG`	0 (reset)	2.0 (via T reset)	Truncated	Pass
`IGNORE_ANOM`	0	default	Excluded	Pass
(none)	0	tiered by T	All kept	**Blocks**

5 Operational Scenarios

The architecture supports four distinct operational scenarios, each exercising different components:

Scenario A: Normal operation. All metrics score **GREEN** or **YELLOW**. The pipeline passes. Data is published. No human intervention required.

Scenario B: Real drift detected. One or more metrics score **RED** with no existing override. The pipeline blocks with `sys.exit(1)`. The auditor downloads the artifact bundle, reviews dashboards in the Streamlit UI, determines the cause, selects an appropriate override, and re-triggers Stage 2.

Scenario C: Explainable regime change. A policy or definitional change produces a legitimate structural shift (e.g., a 27.3% increase in alternative school enrollment due to accountability-model reclassification). The auditor applies `REGIME_CHANGE`, truncating pre-regime history. The pipeline re-runs and passes.

Scenario D: Multi-year policy transition. A new policy produces an initial spike (**RED**), followed by continued drift at a reduced rate. The auditor applies `TRANSITION_STATE` in Year 1 and Year 2, then `REGIME_CHANGE` once stability is confirmed. The κ_0 tiering automatically adjusts sensitivity at each stage of the lifecycle.

Scenario E: Multi-version preliminary cycle. Within a fiscal year, multiple preliminary data versions exercise Stage 1 for code-correctness validation. Overrides from the previous preliminary version are reset (see Section 2.4). Stage 2 is activated

only for the final annual release, where branch-protection policy requires its passage for merge.

6 Illustrative Deployment Scenarios

We illustrate the architecture's behavior using representative scenarios drawn from a deployment against a large-scale administrative data system (millions of records, 80+ monitored metrics, multiple annual epochs). All figures below are representative of the system's operational characteristics; specific values have been adjusted for confidentiality.

True positive: regime change. The system correctly flagged a year-over-year increase exceeding 25% in a structural metric ($p < 0.005$, $z > 3.0$). Investigation traced this to a reclassification of organizational categories—a definitional change rather than a data error. This was resolved via `REGIME_CHANGE`, demonstrating the override taxonomy in practice.

Appropriate tolerance of historical volatility. A subgroup metric exhibited historical changes exceeding +1000% and +100% across consecutive epochs, yet a subsequent large decrease was correctly scored **GREEN** ($|z| < 1.5$), because the posterior predictive had learned this metric's extreme volatility.

Materiality filtering. Organizational-unit counts at the sub-state level—where even the largest unit contains fewer than 100 entities—routinely trigger **YELLOW** or **RED** flags for changes of 1–2 units. The materiality filter correctly identifies these as low-confidence, preventing operationally meaningless alerts from consuming reviewer attention while still surfacing them on the dashboard for optional investigation.

Artifact-injection workflow. The full remediation loop—from pipeline failure to override entry to successful Stage 2 re-run—required no manual identifier transcription. The auditor downloaded the artifact, ran the launcher, reviewed dashboards, selected overrides, and triggered re-evaluation from the Streamlit interface. Selective stage-skipping eliminated redundant Stage 1 computation (approximately 25 minutes of Docker build and multi-million-record processing).

7 Related Work

Data validation frameworks. Great Expectations, Amazon Deequ [Schelter et al., 2018], and Google's data validation tools [Breck et al., 2019] provide rule-based or schema-based validation. These systems excel at structural checks but do not model historical trends or adapt thresholds.

Statistical process control. Shewhart, CUSUM, and EWMA charts [Montgomery, 2009] require 20–25 subgroups for parameter estimation, inapplicable when $T < 5$.

Bayesian monitoring. Adams and MacKay [2007] propose Bayesian online changepoint detection for streaming data. Our setting differs: batch-level aggregates with sparse temporal data rather than high-frequency streams.

Hierarchical Bayesian models. Borrowing strength across related units via hierarchical priors is well-established [Gelman et al., 2013, Efron, 2012]. Our contribution integrates this with CI/CD governance, circuit-breaker patterns, and typed human override protocols.

Write-Audit-Publish. The WAP pattern is advocated in modern data lakehouse architectures for separating data production from quality assurance. Our work provides a principled statistical method for the Audit stage, replacing rule-based checks with adaptive Bayesian scoring. The artifact-injection protocol extends WAP with a closed-loop remediation mechanism that bridges cloud and local environments.

Human-in-the-loop ML governance. HITL patterns are common in model monitoring [Amershi et al., 2019] but are typically advisory. Our circuit breaker is *blocking*: anomalous data cannot reach production without explicit human classification. The four-state override taxonomy pro-

vides structured governance beyond binary accept/reject.

8 Discussion

8.1 Separation of Concerns

The architecture cleanly separates three concerns that are typically conflated in ETL validation:

- **Code correctness** (Stage 1): schema conformance, value-range constraints, referential integrity—properties of the transformation logic that can break within a single fiscal year.
- **Statistical consistency** (Stage 2): year-over-year distributional stability—a property of the data that requires multi-epoch context.
- **Domain judgment** (HITL overrides): classification of anomalies into actionable categories—a property of institutional knowledge that cannot be automated.

This separation means that a code change that passes all Stage 1 tests but introduces a subtle distributional shift (e.g., changing a filter predicate that alters a subgroup proportion by 3%) will be caught by Stage 2, while a policy change that legitimately shifts distributions (e.g., reclassifying school types) can be handled via the override taxonomy without code modifications.

8.2 Applicability Beyond Education

The HVLF regime is common across regulated industries. Insurance claims processing produces annual actuarial tables; government census data arrives decennially; clinical trial endpoints are evaluated at scheduled interim analyses; financial reporting follows quarterly or annual cycles. The architecture requires only: (i) a metric-extraction function per domain; (ii) a SQL backend for ground truth and overrides; (iii) a CI/CD platform with artifact publishing and conditional stage execution.

The statistical model is domain-agnostic—the Normal-Inverse-Gamma conjugate family and log-ratio transform impose no assumptions about the subject matter, only that metrics are positive-valued and approximately log-normally distributed year-over-year.

8.3 Extensibility to Other Conjugate Families

While the current production implementation relies on the Normal-Inverse-Gamma conjugate family applied to log-ratios (accommodating continuous percentage changes), the decoupled architecture proposed in Section 2 is distribution-agnostic. The $O(1)$ Stage 2 scoring module can be trivially extended to accommodate other data-generating processes without altering the CI/CD infrastructure or the Human-in-the-Loop circuit breaker. Table 2 outlines natural extensions for future work, mapping alternative conjugate pairs to specific administrative data types.

Table 2: Architectural Extensibility: Mapping Conjugate Families to Administrative Data Types.

Conjugate Prior / Likelihood	Posterior Predictive	Random Variable Type	Example Use Case
Normal-Inverse-Gamma	Student-t	Continuous (Log-Ratios)	SGP scores, YoY test score deltas
Beta / Binomial	Beta-Binomial	Binary / Proportions	EL status, absenteeism rates
Dirichlet / Multinomial	Dirichlet-Multinomial	Categorical count vectors	Ethnicity subgroups, proficiency levels

9 Limitations

With $T = 2$ historical log-ratios in initial deployment, the prior retains substantial influence. Early-stage results should be interpreted as conservative screening. Detection power improves automatically as T increases.

The hierarchical pooling assumes approximate exchangeability of variance within metric groups. Metrics undergoing structural changes violate this assumption and should be handled via the regime-change or transition-state overrides.

Log-ratios are undefined for zero-valued metrics. Metrics that can take zero values require separate treatment or additive smoothing.

The `TRANSITION_STATE` momentum prior (Section 4.1) is a heuristic that replaces μ_0 with the most recent log-ratio. This is not a principled random-walk model; a formal state-space formulation would require MCMC inference incompatible with the $O(1)$-per-metric constraint.

Flagging thresholds (1% **RED**, 5% **YELLOW**) are operationally motivated. Organizations should calibrate to their cost structure.

Materiality thresholds. The deterministic step-function thresholds (Equation (13)) are

uniform across metric types. In practice, "small count" means different things for different metrics: a region with 16 organizational units may warrant full scoring, while a subgroup with 16 individuals may not. Metric-type-aware thresholds are a natural extension but require domain-specific calibration.

Scalability limits. While Stage 2 Bayesian scoring is strictly $O(1)$ per metric and resolves instantaneously, the ultimate scalability bottleneck of this architecture resides in the Stage 1 I/O throughput. As dataset sizes approach 10^8 records, Stage 1 execution time will scale linearly. Future optimizations for ultra-high-volume regimes may require pushing Stage 1 aggregations down to a distributed query engine (e.g., Spark, Databricks) rather than executing them within the CI agent's local memory space.

The artifact-injection protocol currently targets Azure DevOps. Adaptation to other CI/CD platforms (GitHub Actions, GitLab CI, Jenkins) requires only remapping the context-variable names and API endpoints; the architectural pattern—artifact as relay, local remediation, selective re-triggering—is platform-agnostic.

10 Conclusion

We have described a governance architecture comprising four interlocking components:

1. Two-stage computational decoupling separating $O(N)$ aggregation from $O(1)$-per-metric Bayesian scoring on log-ratios;
2. A hierarchical Normal-Inverse-Gamma model with robust median pooling, adaptive κ_0 tiering for low-history metrics, and deterministic materiality-aware filtering;
3. A human-in-the-loop circuit breaker with a four-state override taxonomy supporting anomaly exclusion, regime truncation, acknowledged outliers, and transition-state momentum priors, with structured lifecycle patterns for multi-year transitions;
4. An artifact-injection protocol that bridges cloud CI agents and local auditor environments, enabling zero-transcription override entry and selective stage re-triggering.

The individual statistical and engineering components are well-known. The contribution is their integration into a cohesive, deployable system evaluated on a deployment processing millions of records across 80+ monitored metrics. The architecture is framework-agnostic and is designed for regulated industries where audit trails and human oversight are non-negotiable.

Code availability. The scoring module and reference implementation are available at https://github.com/kaispace30098.

Acknowledgment

This work was motivated by the author's experience with large-scale administrative data systems. All methodology was developed and evaluated using synthetic data. Architectural designs expressed herein are solely those of the author.

References

Adams, R. P. and MacKay, D. J. C. (2007). Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*.

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. (2019). Software engineering for machine learning: a case study. In *Proc. ICSE-SEIP*, pages 291–300.

Breck, E., Polyzotis, N., Roy, S., Whang, S., and Zinkevich, M. (2019). Data validation for machine learning. *Proc. MLSys*.

Databricks (2023). Write-Audit-Publish pattern. https://docs.databricks.com.

Efron, B. (2012). *Large-Scale Inference: Empirical Bayes Methods for Estimation, Testing, and Prediction*. Cambridge University Press.

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., and Rubin, D. B. (2013). *Bayesian Data Analysis*, 3rd ed. Chapman & Hall/CRC.

Great Expectations (2023). https://greatexpectations.io.

Montgomery, D. C. (2009). *Introduction to Statistical Quality Control*, 7th ed. Wiley.

Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., and Grafberger, A. (2018). Automating large-scale data quality verification. *Proc. VLDB Endowment*, 11(12):1781–1794.

A Reference Implementation

This appendix provides the reference implementation for the artifact-injection protocol described in Section 2.3: the Azure DevOps pipeline YAML (Section A.1), the Streamlit auditor application (Section A.2), and the Docker container definition (Section A.3).

A.1 Azure DevOps Pipeline (Stage 2 Gate)

The pipeline accepts an `AUDITOR_DATA` parameter. On first invocation (no audit data), it packages the auditor application, dashboards, and context metadata into a downloadable artifact and exits with a non-zero code. On re-invocation with audit data (from the Streamlit UI), it proceeds to publish.

Listing 1: Azure DevOps pipeline YAML for the audit gate stage.

```yaml
parameters:
  - name: AUDITOR_DATA
    type: string
    default: 'EMPTY'

trigger: none

pool:
  name: 'Build-Agent-Pool'

stages:
  - stage: UploadTable
    displayName: 'Audit & Package'
    jobs:
      - job: CreateAndPublish
        steps:
          - powershell: |
              $auditData = $env:RAW_DATA
              $internalPat = $env:ADO_PAT_SECRET
              $dir = "$(Build.ArtifactStagingDirectory)\auditor_package"

              if (!(Test-Path $dir)) {
                New-Item -Path $dir -ItemType Directory -Force
              }

              if ($auditData -eq "EMPTY") {
                  Write-Host "No Audit Data found. Packaging files..."

                  # 1. Create pending metrics stub
                  "FiscalYear,Metric,Category`n2025,UnitCount,Type_A" |
                    Out-File -FilePath "$dir\pending_metrics.csv" -Encoding utf8

                  # 2. Copy auditor app files
                  Copy-Item "$(Build.SourcesDirectory)\audit_ui.py" `
                    -Destination "$dir\"
                  Copy-Item "$(Build.SourcesDirectory)\Dockerfile" `
                    -Destination "$dir\"

                  # 3. Generate .bat launcher
                  $bat = @"
@echo off
docker stop auditor-poc >nul 2>&1
docker rm auditor-poc >nul 2>&1
echo Starting Auditor UI...
start http://localhost:8501
docker build -t auditor-poc .
docker run --name auditor-poc -it --rm ^
  -p 8501:8501 ^
  -e ADO_PAT=%ADO_PAT_SECRET% ^
  auditor-poc
pause
"@
                  [System.IO.File]::WriteAllLines(
```

```
                    "$dir\start_auditor.bat", $bat)

                  Write-Host ("##vso[task.logissue type=error]" +
                    "AUDIT REQUIRED: Download artifact " +
                    "'auditor-package' and run start_auditor.bat")
                  exit 1
              } else {
                  Write-Host "Audit Data Received: $auditData"
              }
            displayName: 'Audit Gate Logic'
            env:
              RAW_DATA: ${{ parameters.AUDITOR_DATA }}

          - task: PublishPipelineArtifact@1
            condition: always()
            inputs:
              targetPath: >-
                $(Build.ArtifactStagingDirectory)\auditor_package
              artifactName: 'auditor-package'
```

Note: The original implementation embedded a personal access token directly in the generated batch file. The listing above replaces this with an environment-variable reference (`%ADO_PAT_SECRET%`) sourced from the CI secret store. Production deployments should never embed credentials in artifact files.

A.2 Streamlit Auditor Application

The auditor application reads pipeline context from environment variables, displays pending metrics, collects override decisions, and re-triggers the pipeline via the Azure DevOps REST API.

Listing 2: Streamlit auditor portal (`audit_ui.py`).

```python
import streamlit as st
import pandas as pd
import requests
import json
import os

# --- Configuration ---
ADO_ORG = os.getenv("ADO_ORG", "MYORG")
ADO_PROJECT = os.getenv("ADO_PROJECT", "MyProject")
PIPELINE_ID = os.getenv("PIPELINE_ID", "1233")

# Context injected from artifact
PR_ID = os.getenv("PR_ID", "MANUAL")
PAT = os.getenv("ADO_PAT", "")

# Determine git ref for re-trigger
if PR_ID.isdigit():
    GIT_REF = f"refs/pull/{PR_ID}/merge"
else:
    GIT_REF = "refs/heads/main"

st.set_page_config(page_title="Auditor Portal", layout="wide")
header = f"PR #{PR_ID}" if PR_ID.isdigit() else "Manual Run"
st.title(f"Auditor Portal ({header})")

# --- Display pending metrics ---
try:
    df = pd.read_csv("pending_metrics.csv")
    st.subheader("Metrics Pending Review")
    st.dataframe(df, use_container_width=True)
except FileNotFoundError:
    st.warning("Data file not found.")

st.divider()

# --- Override selection ---
```

```python
OVERRIDE_TYPES = [
    "IGNORE_ANOMALY",
    "REGIME_CHANGE",
    "TRANSITION_STATE",
    "ACKNOWLEDGED_OUTLIER",
]
status_choice = st.selectbox("Select Override Status", OVERRIDE_TYPES)
notes = st.text_area("Audit Notes")

# --- Submit and re-trigger ---
if st.button("Submit & Unlock Pipeline"):
    if not PAT:
        st.error("Missing PAT. Set ADO_PAT environment variable.")
    else:
        audit_payload = json.dumps({
            "status": status_choice,
            "notes": notes,
        })
        url = (
            f"https://dev.azure.com/{ADO_ORG}/{ADO_PROJECT}"
            f"/_apis/pipelines/{PIPELINE_ID}/runs?api-version=7.0"
        )
        payload = {
            "templateParameters": {"AUDITOR_DATA": audit_payload},
            "resources": {
                "repositories": {
                    "self": {"refName": GIT_REF}
                }
            },
        }

        with st.spinner(f"Triggering Pipeline on {GIT_REF}..."):
            resp = requests.post(url, auth=("", PAT), json=payload)
            if resp.status_code in [200, 201]:
                st.success("Success! Re-run triggered.")
                st.balloons()
            else:
                st.error(f"Error {resp.status_code}")
                st.json(resp.json())
```

A.3 Dockerfile

Listing 3: Docker container for the auditor application.

```dockerfile
FROM python:3.9-slim
WORKDIR /app

RUN pip install --no-cache-dir \
    --trusted-host pypi.org \
    --trusted-host pypi.python.org \
    --trusted-host files.pythonhosted.org \
    streamlit pandas requests

COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "audit_ui.py", \
     "--server.port=8501", \
     "--server.address=0.0.0.0"]
```